



파이썬 프로그래밍

자료 구조 2 (Dictionary, Set)

Fall Semester 2025-02

Outline

1. 딕셔너리 (Dictionary)
2. 집합(Set)

딕셔너리(Dictionary)

- 키(key)와 값(value)으로 구성된 한 쌍의 데이터를 담을 수 있는 자료구조 (map, hash라고 부르기도 함)
- 중복된 키가 포함될 수 없음
- 키는 수정 불가능(immutable)한 것만 사용 가능
- 값은 변경 가능

- 중괄호로 표현

```
{ 키1 : 값1, 키2 : 값2 }
```

- 요소에 대한 접근은 [] 사용
 - ▣ 딕셔너리_이름[키] 또는 get(키)함수 사용
- 값 추가 또는 변경
 - ▣ 딕셔너리_이름[키] = 값
- 삭제는 del() 사용

키(key)	값(value)
키1	값1
키2	값2

딕셔너리(Dictionary)

□ 딕셔너리 예

```
{ } # 빈 딕셔너리  
{ "name" : "홍길동", "age" : 20,  
  "취미" : ["영화 감상", "게임", "독서"] }
```

□ 키

- 수정 불가능한 것들만 사용할 수 있음
- 문자열, 숫자, 불린 값, 튜플 등

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
print(d[1])  
print(d[False])  
print(d[(1,2)])
```

딕셔너리(Dictionary)

- 딕셔너리의 요소 개수 확인하기
 - ▣ `len()` 함수 사용

```
d = {}      # empty dictionary
d1 = {'a':1, 'b':2, 'c':3 }
len({})     # 0
len(d)
len(d1)
len({'a':1, 'b':2, 'c':3 })
```

딕셔너리(Dictionary)

- 요소 접근 예 (딕셔너리_이름[키] 사용)

```
d = { "name" : "홍길동", "age" : 20,  
      "취미" : ["영화 감상", "게임", "독서"] }  
print(d["name"]) # 홍길동  
print(d["취미"]) # ["영화 감상", "게임", "독서"]  
print(d["이름"]) # 오류 발생 (KeyError)
```

- 요소 접근 예 (get()함수 사용)

```
d = { "name" : "홍길동", "age" : 20,  
      "취미" : ["영화 감상", "게임", "독서"] }  
print(d.get("name")) # 홍길동  
print(d.get("취미")) # ["영화 감상", "게임", "독서"]  
print(d.get("이름")) # None
```

딕셔너리(Dictionary)

- 요소 값 추가 또는 수정: `dic_name[key]="new value"`
 - ▣ 단일 요소 수정

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
d[1] = 3  
d[False] = "불린 값"  
d[(1, 2)] = [1, 2]  
d["key"] = "value"      # 새로운 키와 값 추가  
print(d[1])  
print(d[False])  
print(d[(1,2)])  
print(d["key"])
```

딕셔너리(Dictionary)

- 요소 값 추가 또는 수정
 - ▣ 여러 요소를 한꺼번에 수정
 - `update()` 함수 사용

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
d.update({1:3, False:"불린 값", (1,2):[1,2],  
         "key":"value" })  
print(d[1])  
print(d[False])  
print(d[(1,2)])  
print(d["key"])
```


딕셔너리

- 키가 딕셔너리에 있는지 확인
 - ▣ In, not in 사용
 - 키와 값이 딕셔너리에 있는지 확인하는 연산자

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
if 1 in d:  
    print(d[1])  
else:  
    print("1은 d의 키가 아닙니다")
```

딕셔너리

□ 비교하기:

□ ==, !=

- 두 개의 딕셔너리가 동일한 항목을 가지고 있는지 확인

□ >, >=, <, <=와 같은

- 비교연산을 지원하지 않는다.

□ 삭제

□ `del(딕셔너리_이름[키])`

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
del(d[1])  
print(d) # { False : 20, (1, 2) : "튜플" }
```

딕셔너리

- 딕셔너리 전체 삭제
 - ▣ clear() 함수 사용

```
d = { 1 : 2, False : 20, (1, 2) : "튜플" }  
d.clear()  
print(d) # {}
```

딕셔너리

- 주의할 점
 - ▣ 키 값으로 리스트 사용 불가



딕셔너리의 메소드

[표 7-3] 딕셔너리의 메소드와 하는 일

메소드	하는 일
keys()	딕셔너리 내의 모든 키를 반환한다.
values()	딕셔너리 내의 모든 값을 반환한다.
items()	딕셔너리 내의 모든 항목을 [키]:[값] 쌍으로 반환한다.
get(key)	키에 대한 값을 반환한다. 키가 없으면 None을 반환한다.
pop(key)	키에 대한 값을 반환하고 그 항목을 삭제한다. 키가 없으면 KeyError 예외를 발생시킨다.
popitem()	마지막으로 입력된 항목을 반환하고 그 항목을 삭제한다.
clear()	딕셔너리 내의 모든 항목을 삭제한다.



집합(set) 자료구조

- 집합set
 - 순서가 없는 자료형
 - 동일한 값을 가지는 항목이 중복이 허용되지 않음
 - 교집합(intersection), 합집합(union), 차집합(difference) 등의 집합 연산 수행 가능

- 집합을 선언하는 방법

[표 7-13] 집합을 만드는 여러 가지 방법

빈 집합 만들기	<code>set0 = set()</code> # <code>set0 = {}</code> 로 사용하면 안 됨
기본적인 집합 만들기	<code>set1 = {1, 2, 3, 4}</code>
튜플로부터 집합 만들기	<code>n_tuple = (1, 2, 3, 4)</code> <code>set2 = set(n_tuple)</code>
리스트로부터 집합 만들기	<code>n_list = [1, 2, 3, 4]</code> <code>set3 = set(n_list)</code>

집합(set) 자료구조

- 집합을 만들 때는 다음과 같이 이미 생성된 리스트나 튜플에 set() 함수를 사용할 수도 있음
- 집합은 순서가 없는 원소의 모임이므로 **인덱스를 사용할 수 없음**



대화창 실습: 리스트와 튜플로부터 집합 만들기

```
>>> days_list = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'] # 리스트
>>> days_set = set(days_list) # 리스트로부터 집합 만들기
>>> days_set
{'Fri', 'Mon', 'Sat', 'Sun', 'Thu', 'Tue', 'Wed'}
>>> fruits_tuple = ('apple', 'orange', 'water melon') # 튜플
>>> fruits_set = set(fruits_tuple) # 튜플로부터 집합 만들기
>>> fruits_set
{'apple', 'orange', 'water melon'}
>>> h_str = 'hello' # 문자열
>>> h_set = set(h_str) # 문자열로부터 집합 만들기
>>> h_set
{'h', 'l', 'e', 'o'}
```

집합의 메소드

[표 7-14] 집합의 메소드와 하는 일

메소드	하는 일
add(x)	집합 내의 특정 x 원소를 추가한다.
discard(x)	집합 내의 특정 x 원소를 삭제한다.
clear()	집합 내의 모든 원소를 삭제한다.
union(s)	s 집합과의 합집합을 구한다. 연산과 동일하다.
difference(s)	s 집합과의 차집합을 구한다. - 연산과 동일하다.
intersection(s)	s 집합과의 교집합을 구한다. & 연산과 동일하다.
symmetric_difference(s)	s 집합과의 대칭 차집합을 구한다. ^ 연산과 동일하다.
issubset(s)	s 집합의 부분집합인가를 구한다. True/False를 반환한다.
issuperset(s)	s 집합의 상위집합인가를 구한다. True/False를 반환한다.
isdisjoint(s)	s 집합과 서로소인가를 구한다. True/False를 반환한다.



집합의 메소드



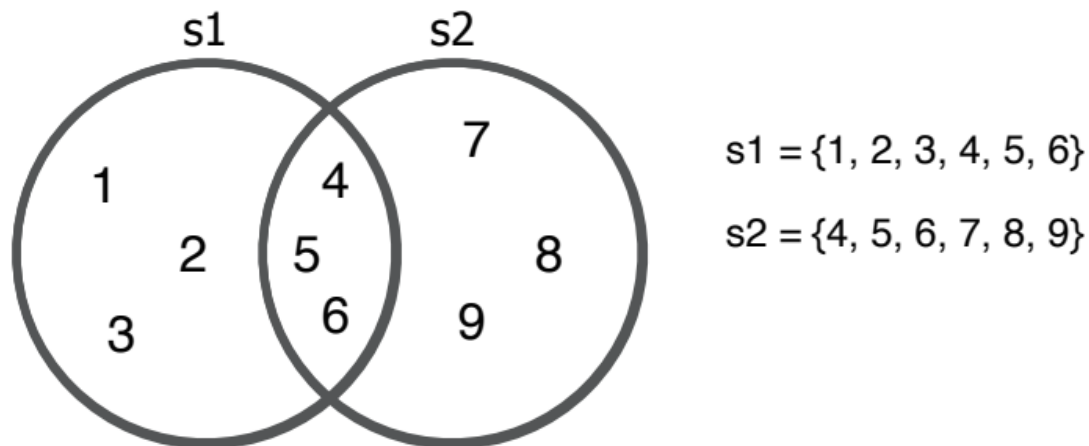
대화창 실습: 집합의 메소드 테스트

```
>>> s = {100, 100, 200, 200, 300, 400} # 중복 원소 100, 200이 있음
>>> s
{100, 200, 300, 400}
>>> s.add(500)      # 집합 s에 500 원소 추가
>>> s
{100, 200, 300, 400, 500}
>>> s.discard(100)  # 집합 s에 있는 100 원소를 삭제
>>> s
{200, 300, 400, 500}
```



집합의 메소드

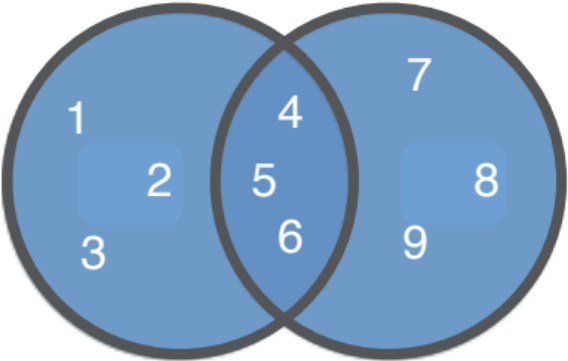
- 교집합은 `&`, 합집합은 `|`, 차집합은 `-`, 대칭 차집합은 `^` 연산을 사용
- 저 기호 대신 `intersection()`, `union()`, `difference()`, `symmetric_difference()` 의 메소드 사용 가능
- 집합 `s1`과 `s2`가 있을 경우 이 집합의 연산을 적용하기 위해서는 벤다이어그램으로 집합의 원소와 포함관계를 표현해야 함



[그림 7-4] 집합 `s1`과 `s2`의 원소들과 포함 관계

집합의 메소드

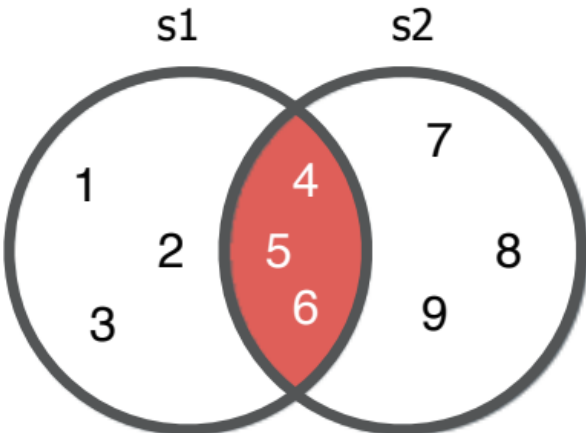
□ 합집합은 | 연산자를 사용(union())

연산	연산 결과 벤다이어그램
합집합	s1 s2 
	$s1 \mid s2 = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ 혹은 <code>s1.union(s2)</code>

[그림 7-5] 집합 s1과 s2의 원소들과 합집합 연산

집합의 메소드

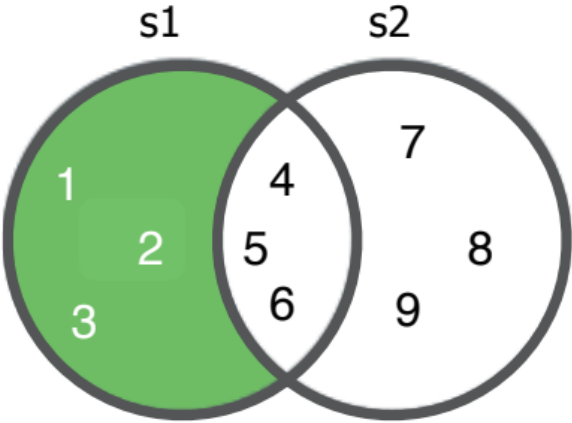
□ 교집합 연산은 & 연산자를 사용(intersection())

연산	연산 결과 벤다이어그램
교집합	
	<code>s1 & s2 = {4, 5, 6}</code> 혹은 <code>s1.intersection(s2)</code>

[그림 7-6] 집합 s1과 s2의 원소들과 교집합 연산

집합의 메소드

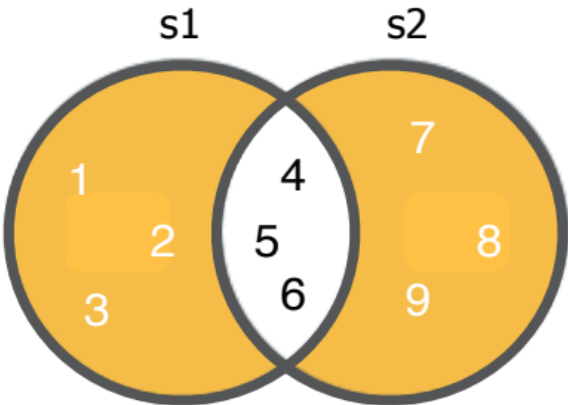
- 차집합 연산은 - 연산자를 사용(difference())

연산	연산 결과 벤다이어그램
차집합	
	$s1 - s2 = \{1, 2, 3\}$ 혹은 <code>s1.difference(s2)</code>

[그림 7-7] 집합 s1과 s2의 원소들과 차집합 연산

집합의 메소드

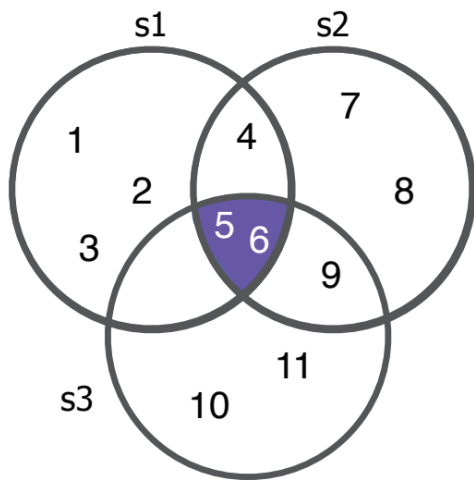
- 대칭 차집합은 s1과 s2의 합집합에서 교집합을 뺀 $(s1 \cup s2) - (s1 \cap s2)$ 연산으로 ^라는 연산 기호를 사용(symmetric_difference())

연산	연산 결과 벤다이어그램
대칭 차집합	 A Venn diagram with two overlapping circles labeled s1 and s2. Circle s1 (left) contains elements 1, 2, 3, 4, 5, 6. Circle s2 (right) contains elements 4, 5, 6, 7, 8, 9. The intersection of s1 and s2 contains elements 4, 5, 6. The symmetric difference of s1 and s2, which is the result of the operation s1 ^ s2, consists of elements 1, 2, 3, 7, 8, 9.
	$s1 \wedge s2 = \{1, 2, 3, 7, 8, 9\}$ 혹은 <code>s1.symmetric_difference(s2)</code>

[그림 7-8] 집합 s1과 s2의 원소들과 대칭 차집합 연산

집합의 메소드

- 여러 개의 집합에 대한 연속적인 연산도 가능



$s1 = \{1, 2, 3, 4, 5, 6\}$

$s2 = \{4, 5, 6, 7, 8, 9\}$

$s3 = \{5, 6, 9, 10, 11\}$

[그림 7-9] $s1 \& s2 \& s3$ 연산의 결과



대화창 실습: 집합과 $s1 - s2 - s3$ 연산

```
>>> s1 = {1, 2, 3, 4, 5, 6}
```

```
>>> s2 = {4, 5, 6, 7, 8, 9}
```

```
>>> s3 = {5, 6, 9, 10, 11}
```

```
>>> s1 & s2 & s3 # s1, s2, s3의 교집합을 구함  
{5, 6}
```

```
>>> s1 - s2 - s3 # s1에서 s2, s3의 원소를 뺀다.  
{1, 2, 3}
```

집합(set) 자료구조

□ 요소 추가

- ▣ add()함수 – 단일 요소 추가

```
a = set([1, 2, 3])  
a.add("string")
```

- ▣ update()함수 – 여러 개 요소 추가

```
a = set([1, 2, 3])  
a.update("string")
```


집합(set) 자료구조

- 요소가 집합에 있는지 확인
 - ▣ in 사용

```
s = { 1, 2, 20, (1, 2), "문자열" }  
if 1 in s:  
    print("1은 집합 s에 포함되어 있습니다")  
else:  
    print("1은 집합 s의 요소가 아닙니다")
```

실습문제 1

□ 문제

- ▣ 1~20 사이의 숫자 중에서 무작위로 중복되지 않는 숫자 10개를 뽑아서 출력

□ 요구사항

- ▣ 무작위 숫자 생성기로 1~20 사이의 정수를 생성한 후 저장
- ▣ 새로운 무작위 숫자를 생성했을 때 기존에 저장되어 있던 숫자들과 비교해서 중복되지 않는 숫자이면 새로 저장하고, 중복된다면 버리고 새로운 숫자를 생성
- ▣ 10개의 중복되지 않는 숫자들이 생성될 때까지 반복. 즉, 무한 반복하면서 10개가 채워지면 반복을 종료해야 함

실습문제 2

□ 문제

- 중복 요소를 가진 튜플 tup에서 중복 요소가 하나만 나타나도록 하여라. 예를 들어, 주어진 튜플에는 2가 두 번 나타나지만, 중복을 제거하시오.
- 주어진 튜플:

1, 2, 5, 4, 3, 2, 1, 4, 7, 8, 9, 9, 3, 7, 3

□ 실행 예시

```
Given tuple: (1, 2, 5, 4, 3, 2, 1, 4, 7, 8, 9, 9, 3, 7, 3)
Tuple with duplicates removed: (1, 2, 3, 4, 5, 7, 8, 9)
```

실습문제 3

□ 문제

- 한도시에 살고 있는 구별 인구 중 10개를 가나다순으로 뽑아서 문자열(guPopulation)을 만들었다.
 - `guPopulation = "강남구,233363,강동구,199088,강북구,144410,강서구,267442,관악구,273736,광진구,166638,구로구,180027,금천구,114402,노원구,217272,도봉구,138120"`
- 이 문자열을 이용해서 구:인구 형태의 딕셔너리를 구성
- 사용자로 부터 정수 한 개를 입력 받고, 입력한 값보다 인구가 적은 구를 찾아 구의 이름과 인구를 화면에 출력하는 프로그램을 작성

□ 요구사항

- 사용자는 정수를 입력할 것이라고 가정
- 만약 사용자가 입력한 정수값(num)보다 작은 구가 없으면 "인구가 num보다 작은 구가 없습니다" 를 출력
- 구:인구 형태로 출력

□ 실행 예시

```
Enter the population to compare: 140000
District with a population less than 140000:
금천구:114402
District with a population less than 140000:
도봉구:138120
```

```
Enter the population to compare: 110000
There is no district with a population less than 110000
```



실습문제 4

□ 문제:

- 동윤이가 새로 문을 열고 운영하는 커피 가게에는 Americano, Ice Americano, Cappuccino, Caffe Latte, Espresso의 5가지 메뉴가 있으며, 각 메뉴의 가격은 각각 3,000원, 3,500원, 4,000원, 4,500원, 3,600원이다. 이 목록을 menu라는 딕셔너리로 작성하라.

- 이 menu의 내용을 for key in menu: 구문을 사용하여 다음과 같이 출력하여라.

```
Americano      Price: 3,000 won
Ice Americano   Price: 3,500 won
Cappuccino      Price: 4,000 won
Cafe Latte      Price: 4,500 won
Espresso        Price: 3,600 won
```

- 카페 손님으로부터 다음과 같은 주문을 받은 후 주문한 내용이 메뉴에 있는지 구하는 프로그램을 작성하라

```
Please select one of the menus above: Americano
Americano is 3,000 won. Please pay.
```

```
Please select one of the menus above: Orange juice
Sorry, Orange juice is not in the menu.
```

실습문제 5

□ 문제

- ▣ 튜플을 요소로 가지는 student_tuple 튜플이 다음과 같이 있다. 이 튜플의 요소가 되는 튜플은 (학번, 이름, 전화번호)로 이루어져 있다. 이를 이용하여 (학번:이름)의 딕셔너리를 만들고 학번을 입력하면 학생의 이름이 나오도록 하라.

- `student_tuple = (('191101', 'Jhon', '010-123-45xx'), ('191102', 'Mina', '010-223-45xx'), ('191103', 'Lia', '010-323-45xx'))`

□ 요구사항

- ▣ 사용자에게 반복적으로 학번을 입력받을 것.
- ▣ 다음 조건을 충족하는 프로그램 작성:
 - 입력된 학번이 딕셔너리에 존재하면:→ "Student number XXXX is NAME." 출력
 - 입력된 값이 음수이면:→ "Quit the program." 출력 후 프로그램 종료
 - 그외 번호 입력 시:→ "There is no student with that number." 출력

실행
예시

```
Student Information: {'191101': 'Jhon', '191102': 'Mina', '191103': 'Lia'}
Enter your student number: 191101
Student number 191101 is Jhon.
Enter your student number: 19
There is no student with that number.
Enter your student number: 191102
Student number 191102 is Mina.
Enter your student number: -1
Quit the program.
```

실습문제 6

□ 문제

- (학생의 이름, 국어, 수학, 과학 성적)이 하나의 튜플을 이루는 데이터가 있으며, 이렇게 튜플로 표시된 학생의 성적을 요소로 하는 튜플 `scores`가 아래와 같이 있다.
- `scores = ( ('Jhon', 88, 95, 90), ('Mina', 85, 90, 95), ('Lia', 70, 90, 80), ('Peter', 90, 90, 95))`
- 이러한 튜플에 대하여 학생의 이름과 국어, 수학, 과학 성적의 평균 점수를 `student_dic`이라는 딕셔너리 구조에 넣은 후 다음과 같이 이름과 평균 성적을 출력하여라.

□ 실행 예시

Name	Average Grade

Jhon	91.00
Mina	90.00
Lia	80.00
Peter	91.67

실습문제 7

- 문제: 수강 과목 조회 프로그램
 - ▣ 다음과 같이 학생별로 수강 중인 과목이 딕셔너리(Dictionary) 와 집합(Set) 형태로 주어져 있다:
 - `students = { "Jhon": {"Math", "Physics"}, "Mina": {"English", "Math"}, "Lia": {"Biology"}, "Alex": {"Math", "Biology", "English"} }`
- 요구사항
 1. 사용자로부터 과목명을 입력받아라.
 2. 입력된 과목을 수강하는 학생들의 이름을 집합(Set)으로 만들어 출력하라.
 3. 해당 과목을 수강하는 학생이 없으면 "No students found."를 출력하라.
 4. 그 외의 경우, 해당 과목을 수강하는 학생들의 이름을 출력하라.
- 실행 예시

```
Enter a course name: Math
Students taking Math: {'Jhon', 'Alex', 'Mina'}
```

```
Enter a course name: Programming
No students found.
```


Challenge

- 문제: 대학 행사 참여 분석 프로그램
 - ▣ 학교에서 여러 행사가 열렸고, 각 행사마다 참석한 학생들의 ID가 기록되어 있다. 다음은 행사별 참석자 정보가 담긴 딕셔너리이다:
 - ▣ `event_attendance = {"Seminar": ["S01", "S02", "S03", "S04"], "AI Workshop": ["S02", "S03"], "Job Fair": ["S01", "S03", "S05"], "Hackathon": ["S03", "S04", "S06", "S02"]}`
- 요구사항
 1. 모든 참석자 리스트를 집합(Set)으로 변환하라.
 2. 다음 항목을 계산하라:
 1. 모든 행사에 참석한 학생들 (교집합)
 2. 하나 이상의 행사에 참석한 학생들 (합집합)
 3. `attendance_count` 라는 딕셔너리를 만들어라:
 1. `key` = 학생 ID `value` = 해당 학생이 참석한 행사 수
 4. 행사에 가장 많이 참석한 학생을 기준으로 상위 2명을 출력하라.
- 실행 예시

```
== Converted to a dictionary with sets ==  
{ 'Seminar': {'S01', 'S03', 'S04', 'S02'}, 'AI Workshop': {'S03', 'S02'}, 'Job Fair': {'S01', 'S03', 'S05'}, 'Hackathon': {'S03', 'S04', 'S06', 'S02'}}
```

```
Students who attended ALL events: {'S03'}
```

```
Students who attended ANY event: {'S05', 'S02', 'S01', 'S03', 'S06', 'S04'}
```

```
Attendance count: {'S01': 2, 'S03': 4, 'S04': 2, 'S02': 3, 'S05': 1, 'S06': 1}
```

```
Top 2 most active students: [('S03', 4), ('S02', 3)]
```